

UN TOKEN QUE TODO EL MUNDO YA SABE **HABLAR**

Construir FichaUSB sobre OpenZeppelin, desplegarla y verificarla en Sepolia, importarla en MetaMask, transferir, y dominar approve / transferFrom con revocación.

MODALIDAD	REQUISITO	META	ENTREGA
En parejas 90 minutos	Repo de laboratorios + MetaMask (curso) en Sepolia	12 pruebas verdes + token verificado en Sepolia	Token, hashes de transferencia y de canje, tokenómica

LA REGLA DE ORO

Todo ocurre en **Sepolia** (red de prueba) o en la red local, nunca con dinero real. La clave privada es la de la **billetera del curso**, sin fondos reales, y vive **solo en el almacén cifrado** de Hardhat (`keystore`), jamás en un archivo del repositorio. La frase semilla no se comparte con nadie. Estos contratos son material de clase **sin auditar**.

01 Qué vas a construir, y por qué

LEER PRIMERO

Un token ERC-20 no vale por tener el mejor código: vale porque **miles de aplicaciones ya saben hablarle**. Ese acuerdo —seis funciones y dos eventos con nombres fijos— es el estándar. Hoy construyes uno **sobre OpenZeppelin** (auditado), no a mano, y aprendes la mecánica de aprobaciones que usa todo DeFi.

PIEZA	QUÉ HACE	ESTÁNDAR QUE APLICA
FichaUSB	Token de puntos de un evento, con tope, acuñación controlada y quema.	ERC20 + Capped + Burnable + Ownable
acunar	El organizador crea puntos, hasta el tope.	<code>_mint</code> con control de acceso
Canje	Un puesto que cobra fichas por un premio usando la aprobación.	approve + transferFrom

EL PUNTO DE HOY, EN UNA FRASE

Un token es una **tabla de saldos dentro de UN contrato**. Tu billetera no «contiene» tokens: contiene la clave que autoriza a mover una fila de esa tabla. Por eso, cuando despliegues el token, **no aparecerá solo** en MetaMask: habrá que importarlo con la dirección del contrato (paso 8).

02 Preparar el entorno

UNA SOLA VEZ

Todo vive en el repositorio único de laboratorios, `material/laboratorios-evm` (Hardhat 3.16, Solidity 0.8.34, OpenZeppelin 5.6.1). Si es la primera vez en esta máquina:

```
cd material/laboratorios-evm
npm install
npx hardhat test          # confirma que el entorno está sano
```

EL FLUJO DEL LABORATORIO: TE GUÍAN LAS PRUEBAS

Recibes un **andamiaje** (en `andamiaje/s10/`) con los dos contratos incompletos: compilan, pero cuatro `// TODO` están por escribir. Las cuentas dicen cuándo terminaste. Copia el andamiaje sobre la carpeta de trabajo y corre las pruebas:

```
copy andamiaje\s10\*.sol contracts\s10\      (Windows)
cp andamiaje/s10/*.sol contracts/s10/        (macOS / Linux)

npx hardhat test test/s10/FichaUSB.test.js
```

Al empezar, varias de las **12 pruebas fallan**. Vas completando los `TODO` y volviendo a correr; el laboratorio está terminado cuando las 12 pasan.

Casi todo lo hereda OpenZeppelin. Tú solo escribes las **decisiones de diseño**. Cada TODO trae su pista en el propio archivo; aquí está el porqué.

1

TODO 1 · el tope en el constructor

Convertir tokens a unidades mínimas

El problema: el tope llega EN TOKENS (p. ej. 1 000 000), pero `ERC20Capped` lo espera en **unidades mínimas** (1 token = 10^{18} unidades).

Qué escribir: multiplica el parámetro por `10 ** decimals()` al pasarlo a `ERC20Capped(...)`.

Por qué: la EVM no tiene decimales reales (Sesión 5). Si no conviertes, tu «tope de un millón» es en realidad un millón de *billonésimas* de token: una fracción invisible.

2

TODO 2 · la función acunar

Crear tokens, en unidades mínimas

Qué escribir: llama a `_mint(a, cantidadEnTokens * 10 ** decimals())`.

Pistas: `_mint` es la función interna de OpenZeppelin que crea tokens. No compruebes el tope aquí:

`ERC20Capped` ya lo hace en `_update` (TODO 3). El modificador `onlyOwner` (de `Ownable`) ya restringe quién puede llamarla.

3TODO 3 · el `_update` que compone

Resolver la herencia múltiple

Qué escribir: una sola línea, `super._update(from, to, value);`.

Por qué existe: al heredar de `ERC20` y de `ERC20Capped`, las dos definen `_update`. Solidity te obliga a decir cuál se usa (por eso el `override(ERC20, ERC20Capped)`). `super._update` llama primero al de `Capped` (que revisa el tope) y luego al de `ERC20` (que mueve los saldos).

Cómo lo notas: mientras esté vacío, **ninguna transferencia mueve saldos**: casi todas las pruebas fallan.

4

TODO 4 · la función canjear

Retirar lo aprobado, ni un token más

Qué escribir, en orden:

a) Mover `precio` fichas **desde** `msg.sender` **hacia** `tesoreria`. ¿Cuál de las seis funciones del estándar mueve tokens de OTRA cuenta? (`transferFrom`).

b) Esa función devuelve un `bool`: si es `false`, `revert TransferenciaFallida()`.

c) Suma 1 a `premiosDe[msg.sender]`.

d) Emite `Canjeado(msg.sender, precio)`.

Pregunta para pensar: ¿por qué aquí NO escribes ningún `require` sobre la `allowance`? Pruébalo sin aprobar y mira el error: `transferFrom` ya revierte solo si no autorizaste lo suficiente.

LAS CUATRO PRUEBAS ★ SON LA TESIS DE LA SESIÓN

No acuñar sobre el tope · quemar libera espacio bajo el tope · el canje retira exactamente lo aprobado · el contrato NO puede llevarse más de lo aprobado. Si esas cuatro pasan, entendiste ERC-20 y el patrón de aprobación. Si te pierdes, vuelve a copiar el andamiaje y empieza de nuevo.

04 Decimales: el número que confunde a todos

ENTENDER ANTES DE DESPLEGAR

Un token con 18 decimales guarda internamente enteros enormes; la coma es pura presentación. Tenlo claro antes de tocar Sepolia, porque los errores de decimales son silenciosos.

LO QUE VE LA PERSONA	LO QUE GUARDA EL CONTRATO
1 FUSB	1 000 000 000 000 000 000 (1×10^{18})
25 FUSB (el precio del canje)	25 000 000 000 000 000 000
0,5 FUSB	500 000 000 000 000 000
1 unidad mínima	1 (indivisible; 0,000000000000000001 FUSB)

En pruebas y scripts se usa `ethers.parseUnits("25", 18)` para escribir y `ethers.formatUnits(valor, 18)` para leer. **Ojo:** no todos los tokens usan 18 decimales —USDC usa 6, así que 25 USDC son `25000000` —; por eso siempre se lee `decimals()` antes de calcular.

05 Guardar los secretos (keystore)

ANTES DE SEPOLIA

Para desplegar en una red pública hacen falta tres datos. Se guardan en el **almacén cifrado** de Hardhat, no en archivos. La primera vez te pide una contraseña para el almacén:

```
npx hardhat keystore set SEPOLIA_RPC_URL      # la URL de tu nodo (Infura/Alchemy)
npx hardhat keystore set SEPOLIA_PRIVATE_KEY  # la clave de la billetera DEL CURSO
npx hardhat keystore set ETHERSCAN_API_KEY    # para verificar el código
```

TRES REGLAS QUE NO ADMITEN EXCEPCIÓN

1. La clave privada es la de la **billetera del curso**, jamás una con fondos reales.
2. Ninguna clave, ninguna URL con API key ni la frase de recuperación se escribe en un archivo del repositorio.
3. Antes de cada `git push`: `git status`, y confirmar que no se sube ningún `.env`, clave ni URL.

06 Desplegar y verificar en Sepolia

CON IGNITION

Se usa **Hardhat Ignition** (como en la Sesión 8): registra cada paso, así que si la conexión se corta, al repetir el *mismo* comando retoma donde quedó y no despliega dos veces. Primero, un ensayo en la red local (no gasta nada):

```
npx hardhat ignition deploy ignition/modules/s10-FichaCanje.js
```

Luego, en Sepolia, con tus parámetros y verificación. Crea el archivo `ignition/parametros/s10-mi-ficha.json` con el tope, el precio (en unidades mínimas) y la dirección de tu segunda cuenta como tesorería:

```
{
  "FichaCanjeModulo": {
    "topeEnTokens": 1000000,
    "precioEnUnidadesMinimas": "25000000000000000000",
    "tesoreria": "0x...tu segunda cuenta..."
  }
}
```

```
npx hardhat ignition deploy ignition/modules/s10-FichaCanje.js \
  --network sepolia \
  --parameters ignition/parametros/s10-mi-ficha.json \
  --verify
```

QUÉ ESPERAR

Ignition despliega **FichaUSB** y, con su dirección, **Canje**. Al final imprime las dos direcciones. `--verify` sube el código fuente a Etherscan: en unos segundos, la pestaña «Contract» del explorador mostrará el código con el check verde. **Si la verificación falla por tiempo**, espera un minuto y repite el *mismo* comando: Ignition sabe que ya desplegó y solo reintenta la verificación. Anota la dirección de **FichaUSB**: la necesitas para el paso siguiente.

1

IMPORTAR EL TOKEN

Para que MetaMask lo «vea»

En MetaMask, con Sepolia activa: «**Importar tokens**» → pega la **dirección del contrato** FichaUSB. El símbolo (FUSB) y los decimales (18) se rellenan solos. Recuerda el paso 1: el token no aparece solo porque no vive en tu billetera, vive en el contrato.

2

ACUÑAR Y TRANSFERIR

Mover puntos entre compañeros

El organizador (la cuenta que desplegó) **acuña** puntos a un compañero, desde la pestaña «Write Contract» del explorador o con un script. El compañero los ve al importar el token. Luego, una **transferencia** normal entre dos cuentas del curso desde MetaMask. **Guarda el hash** de esa transferencia: es evidencia.

3

APROBAR Y CANJEAR

El patrón de dos pasos, en vivo

- a) Aprobar:** desde la cuenta que tiene fichas, llama `approve(<dirección de Canje>, 25000000000000000000)` — el precio EXACTO, 25 FUSB en unidades mínimas. **No** apruebes una cantidad ilimitada (ver paso 9).
- b) Canjear:** llama `canjear()` en el contrato Canje. Este retira las 25 fichas con `transferFrom` y te da un premio.
- c) Comprobar:** mira `allowance(tuCuenta, Canje)`: quedó en **0**, la autorización se gastó. Guarda los hashes de `approve` y de `canjear`.

4

REVOCAR

Recoger la llave que diste

- a)** Aprueba de nuevo el precio (`approve(Canje, 25...)`).
- b) Revoca:** pon la autorización en cero con `approve(Canje, 0)`. Revocar es exactamente esto.
- c) Comprueba:** intenta `canjear()` otra vez: **revierte** con `ERC20InsufficientAllowance`. Guarda el hash del intento fallido (o la captura del error): demuestra que la revocación funciona.

POR QUÉ TRANSFERIR EN DOS PASOS

`transfer` mueve TUS tokens. Pero para que un **contrato** (un canje, un exchange) mueva los tuyos sin tu clave privada, primero lo **autorizas** (`approve`) por una cantidad acotada, y luego él **retira** (`transferFrom`) hasta ese límite, ni un token más. Así funciona todo DEX: primero autorizas al contrato, después el contrato mueve tus tokens al hacer el swap.

08 El riesgo de las aprobaciones ilimitadas

UN CASO REAL

Muchas apps piden aprobar una cantidad gigante «para no molestar otra vez». Es cómodo, y es el origen de robos enormes.

BADGER DAO · DICIEMBRE DE 2021 · ~120 MILLONES DE DÓLARES

El front-end de Badger DAO fue comprometido: un script malicioso inyectado a través de Cloudflare pedía a los usuarios firmar aprobaciones **ilimitadas**. Unas 500 billeteras firmaron sin mirar; el atacante usó `transferFrom` y drenó del orden de **120 millones de dólares**. Los contratos estaban bien: el robo entró por una aprobación excesiva que la gente aprobó a ciegas.

Regla para el proyecto y para la vida: una aprobación es una llave que se queda con otro. Se entrega la más pequeña que sirva, y se recoge (allowance a cero) cuando ya no se usa. Antes de firmar cada `approve`, mira **a quién** autorizas y **cuánto**. Puedes ver y revocar tus aprobaciones activas con herramientas como `revoke.cash` o el «Token Approvals» del explorador (funcionan también en testnet).

09 Bloque C · ¿tu proyecto lleva token?

PLANTILLA DE DECISIÓN

Muchos proyectos **no necesitan** un token. Decidir eso con honestidad vale tanto como diseñar uno bueno. Completa esta plantilla corta (media página) para el Avance 1. Las dos respuestas —lleva o no lleva— valen igual; lo que no vale es un token decorativo.

PREGUNTA	TU RESPUESTA
1. ¿Qué circula entre usuarios y por qué debe ser fungible (cada unidad idéntica a otra)?	
2. ¿Una simple variable en un contrato lo resolvería sin token? Si sí, no lleva token.	
3. ¿La interoperabilidad con otras apps (billeteras, exchanges) aporta de verdad, o es adorno?	

Si la respuesta es **SÍ lleva token**, diseña su tokenómica y justifícala:

DECISIÓN	TU ELECCIÓN Y POR QUÉ
Suministro	Fijo (tope) o inflacionario. ¿Qué comunica?
Emisión	¿Quién acuña y bajo qué regla? («El dueño imprime cuando quiere» es señal de alarma.)
Quema	¿Se pueden destruir tokens? ¿Quién?
Distribución	¿Cómo se reparte el primer lote? ¿Hay vesting?

Si la respuesta es **NO lleva token**, escribe por qué en media página. Un diseño sostenible reparte y limita el poder de emisión; un esquema extractivo concentra ambos en quien lo lanzó. Es el mismo criterio de pertinencia de la Sesión 1.

10 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
Desplugué el token pero no lo veo en MetaMask	No falló nada. Hay que importarlo con la dirección del contrato (paso 7.1). El token vive en el contrato, no en tu billetera.
acunar(ana, 100) y aparece una fracción minúscula	Olvidaste multiplicar por <code>10 ** decimals()</code> (TODO 1/2). La función habla en tokens; el contrato, en unidades mínimas.
La transferencia revierte con ERC20InsufficientBalance	Intentas mover más de lo que tienes. Revisa el balance en unidades mínimas.
canjear() revierte con ERC20InsufficientAllowance	No aprobaste, aprobaste menos que el precio, o ya gastaste/revocaste la aprobación. Vuelve a <code>approve</code> el precio exacto.
El contrato no compila: «_update no override»	Falta el <code>_update</code> con <code>override(ERC20, ERC20Capped)</code> (TODO 3). Al mezclar ERC20 y Capped hay que resolver cuál gana.
La verificación en Etherscan falla	Falta <code>ETHERSCAN_API_KEY</code> en el keystore, o hay que esperar un minuto y repetir el mismo comando de despliegue.
«acunar» revierte con OwnableUnauthorizedAccount	Estás llamando desde una cuenta que no es el dueño. Solo el organizador (quien desplegó) acuña.

11 Preguntas de reflexión

MIRA EL ARTEFACTO REAL

1. Entra a tu FichaUSB verificada en `sepolia.etherscan.io`. En la pestaña «Read Contract», lee `totalSupply` y `cap`. ¿Cuántas unidades mínimas son? ¿Cuántos tokens? Explica por qué los dos números difieren en 18 ceros.
2. Mira, en el explorador, el evento `Approval` de tu `approve` y el `Transfer` de tu `canjear`. ¿Por qué el canje generó un `Transfer` pero no un segundo `Approval`? ¿Qué le pasó a la `allowance`?
3. Después de revocar (`allowance` a 0), intenta canjear y falla. Explica, con esto delante, por qué una aprobación ilimitada que *no* se revoca es exactamente la puerta por la que entró el robo de Badger DAO.

12 Lista de chequeo · rúbrica · referencias

ANTES DE ENTREGAR	DETALLE	PESO
Token verificado en Sepolia, importado en la billetera.	Dirección + captura del check verde.	35 %
Transferencia entre dos cuentas del curso.	Hash de la transacción.	20 %
Canje completo con revocación.	Hashes de <code>approve</code> , <code>canjear</code> y el intento fallido.	30 %
Tokenómica del proyecto (plantilla de la sección 9).	Media página.	15 %

Todo con hashes verificables en el explorador. **Un token que no aparece en la cadena no se califica** (política de laboratorios del curso).

- ERC-20 = EIP-20 (Vogelsteller y Buterin, 2015): <https://eips.ethereum.org/EIPS/eip-20>
- EIP-2612 Permit (Lundfall, 2020): <https://eips.ethereum.org/EIPS/eip-2612>
- OpenZeppelin ERC20: <https://docs.openzeppelin.com/contracts/5.x/erc20>
- Revisar y revocar aprobaciones: <https://revoke.cash>